AG · Caixeiro Viajante — cola

Formato: **o quê** → **por quê**. Valores do código atual (k = 5, população mínima 150).

1. A ideia em 3 linhas

• Problema: passar por **todos** os pontos, uma vez cada, e **voltar ao início**, com a menor distância.
• Testar tudo é impossível: (N−1)!/2 rotas (20 pontos ≈ 60 quatrilhões).
• AG = imitar a evolução: rotas boas viram pais, filhos misturam os pais, às vezes mutam. Acha uma rota **boa**, não garante a ótima.

2. Uma época (o laço)

Antes: **gerar pontos** → **criar P rotas aleatórias** → **avaliar** → **achar a melhor**

Cada época:

① **elitismo**: copia a melhor para a nova geração
② até encher: **torneio** (2 pais) → **OX1** (filho) → **inversão** (30%)
③ **avalia** a nova, **libera** a antiga
④ **parar?** estagnou ou chegou no limite → fim

3. Parâmetros

Gene	vetor de índices (permutação)
População	max(150, 2·N), começa aleatória
Seleção	torneio, k = 5
Cruzamento	OX1, sempre (100%)
Mutação	inversão, 30% por filho
Elitismo	1 (a melhor nunca piora)
Aptidão	1 / distância do ciclo
Parada	max(150, 2·N) épocas sem melhorar, ou max(1000, 20·N) épocas

4. Itens do enunciado

Gene (representação) — 2 pts com o cruzamento

O quê vetor de N números; cada número é um ponto; a ordem é a ordem da visita.
Por quê é sempre uma permutação (sem repetir, sem faltar), então nunca existe rota inválida.

```
[3, 0, 4, 1, 2]  =  3 → 0 → 4 → 1 → 2 → volta ao 3
```

Cruzamento OX1

O quê copia um trecho do pai A; completa com a ordem do pai B, pulando os repetidos.
Por quê o filho sai sempre válido; cruzamento comum repetiria cidades.

```
Pai A:  0 1 [2 3 4] 5 6 7
Pai B:  3 7 5 1 6 0 2 4
Filho:  1 6 [2 3 4] 0 7 5
(lê o pai B a partir da posição 5, dando a volta)
```

Seleção por torneio (k = 5)

O quê sorteia 5 rotas; a mais curta vira pai. Repete para o 2º pai.

Por quê não depende da escala da aptidão (como a roleta) e ninguém domina tudo. k maior = mais pressão, menos diversidade.

Mutação por inversão — 1 pt

O quê 30% de chance por filho: inverte um trecho da rota.

Por quê inverter troca só 2 arestas e desfaz cruzamentos (cruzar = rota mais longa). Taxa baixa trava em ótimo local; diminuir a taxa com o tempo foi testado e piorou. O elitismo protege a melhor.

```
antes:  0 1 [2 3 4 5] 6 7
depois: 0 1 [5 4 3 2] 6 7
```

Aptidão — 1 pt

O quê fitness = 1 / distância. Distância = soma de ponto a ponto + a volta do último ao primeiro.

Por quê minimizar distância = maximizar aptidão. O código compara direto a menor distância (mesma ordem, sem divisão). Sem a volta, seria caminho aberto.

```
quadrado: em volta = 4,00   cruzado = 4,83
```

População e parada — 1 pt

O quê população max(150, 2·N); para com max(150, 2·N) épocas sem melhorar ou max(1000, 20·N) épocas.

Por quê o problema explode com N: com 100 fixos o AG travava a partir de 100 pontos. Mapa maior demora mais entre melhoras. O piso de 150 dá diversidade nos mapas pequenos.

Desempenho a cada época — 2,5 + 2,5 pts

O quê console imprime época, melhor distância, tempo e rota (com X = 1, todas as épocas); a GUI desenha ao vivo.

Por quê círculo tem ótimo conhecido = 2·N·R·sin(π/N), então dá para medir o gap = (AG − ótimo) / ótimo.

5. Minha parte (Pessoa 2)

Memória em bloco único

O quê 3 mallocs: struct + vetor de indivíduos + 1 bloco com todas as rotas.

Por quê em vez de P + 2 mallocs: mais rápido, não vaza pela metade, contíguo. Importa porque cria população nova a cada época.

route_block

O quê guarda onde o bloco começa; o free usa ele.

Por quê se alguém ordenar a população (qsort), path[0] não aponta mais para o começo e o free quebraria.

Fisher-Yates

O quê de i = N−1 até 1: sorteia j entre 0 e i e troca.

Por quê toda ordem tem a mesma chance (1/N!). Trocar pares aleatórios tem viés.

Elitismo com memcpy

O quê copia o conteúdo da rota, não o ponteiro.

Por quê ponteiro copiado = duas rotas na mesma memória → uma estraga a outra e dá double free.

6. Se ele rodar o programa

```
./genetic_tsp_solver 1 10 1 42      (1 = uniforme, 10 pontos, toda época, semente 42)
./genetic_tsp_solver 2 20 10 42     (2 = círculo, imprime a cada 10 épocas)
./genetic_tsp_solver                (sem nada = interface gráfica)
```

Semente? mesma semente = mesmo mapa e mesmo resultado. Sem ela, usa o relógio.

Por que a distância repete várias épocas? nenhum filho foi melhor; o elitismo mantém a melhor. Ela nunca sobe.

Achou na época 5 e parou na 155? 155 = 5 + 150 épocas de estagnação.

"Tempo até o ótimo"? é o tempo até a melhor encontrada; só é o ótimo se o gap for 0.

A rota mudou mas a distância é igual? é o mesmo ciclo começando em outro ponto ou ao contrário.

Rodei de novo e mudou? outra semente = outro mapa e outros sorteios.

Gap (GUI)? só no círculo: (melhor − ótimo) / ótimo.

Velocidade ger/frame (GUI)? só muda a animação, não o algoritmo.

7. Números para lembrar

Círculo	ótimo exato até 200 pontos (200 pontos em ~5 s)
Aleatório	10 pontos: ótimo exato · 30 a 100 pontos: até ~10% acima da referência
Bônus 500	630,83 contra ótimo 628,31 → gap 0,40%, em 122 s
Testes	make test: 38 verificações, 0 falhas

8. Perguntas-armadilha

Garante o ótimo? Não. É heurística ("solucionar localmente").

Por que o círculo é mais fácil? cruzamento lá é óbvio; o aleatório tem muitos ótimos locais.

Por que começar aleatório? diversidade; se começassem parecidas, não haveria o que misturar.

Custo de uma época? ≈ P · N (cada filho custa N no OX1, na inversão e na avaliação).

Quem fez o quê? P1: torneio, OX1, inversão · P2: pontos, população, distância · P3: laço, parada, tempos.

Se travar: explique o **conceito** — o que faz e por quê — mesmo sem lembrar o código.

9. Conceito-chave: exploração × explotação

Explotação = aproveitar o que já é bom: seleção (torneio), elitismo.

Exploração = procurar coisa nova: mutação, população grande, início aleatório.

Explotação demais → a população fica igual cedo e trava num ótimo ruim (convergência prematura).
Exploração demais → vira busca aleatória.

Os parâmetros são o equilíbrio entre as duas: **k = 5**, **mutação 30%**, **população max(150, 2·N)**.

10. "E se...?"

Mutação 0%	só o cruzamento, que mistura o que já existe → a população fica igual e trava.
Mutação 100%	todo filho muda; ainda funciona (inversão é pequena e o elitismo protege), mas perde filhos bons.
Sem elitismo	a melhor rota pode se perder → a curva pode subir.
k = 1	seleção vira sorteio: nenhuma pressão.
k = população	quase sempre vence o melhor → a diversidade some.
População pequena	converge rápido, mas para um ótimo pior.

11. "Por que não...?"

PMX ou CX em vez de OX1?	os três geram rotas válidas. O OX1 preserva um trecho contínuo de rota e a ordem relativa do outro pai; é simples e O(N).
Troca (swap) em vez de inversão?	trocar duas cidades mexe em até 4 arestas e não desfaz cruzamentos; a inversão mexe em só 2 e desfaz.
Roleta?	depende da escala da aptidão; um indivíduo muito bom domina tudo.
Ranking?	funcionaria; o torneio dá efeito parecido sem precisar ordenar a população.